

GUÍA DE TRABAJO

N.º 02 Ciclos, Condicionales y Listas

Profesor: Guillermo Cid Ampuero

Indicaciones generales

- Resuelva cada ejercicio en un archivo independiente utilizando un formato de nombre claro, por ejemplo: `ejercicio_1.py`, `ejercicio_2.py`, etc.
- Mantenga el estándar de importación de librerías: utilice `import numpy as np` e `import matplotlib.pyplot as plt` cada vez que sean necesarias.
- Cuando el objetivo del problema sea implementar un método numérico desde cero, no utilice funciones integradas de Python (como las de `scipy`) que entreguen la solución de forma directa, a menos que se solicite explícitamente para comparar resultados.
- Priorice la claridad en su código. Utilice comentarios para explicar la lógica de su solución, defina funciones propias cuando sea pertinente y nombre sus variables de forma descriptiva.
- En todos los gráficos generados incluya obligatoriamente: título, nombre de los ejes, unidades de medida, cuadrícula (*grid*) y, cuando exista más de una curva, una leyenda.
- Valide sus resultados: antes de ejecutar un programa, analice el problema e intente anticipar el orden de magnitud o el comportamiento esperado de la solución.

Ejercicios

Ejercicio 1 – Filtrado de números pares e impares

Se desea realizar un primer procesamiento de datos utilizando un arreglo de NumPy. Para ello, cree un arreglo con los números enteros desde 0 hasta 20 mediante `np.arange()`. El programa debe realizar lo siguiente:

- Importe NumPy mediante `import numpy as np`.
- Cree el arreglo
$$\text{numeros} = [0, 1, 2, \dots, 20]$$
utilizando `np.arange()`.
- Cree dos listas vacías llamadas `pares` e `impares`.
- Recorra los elementos del arreglo utilizando un ciclo `for`.
- Para cada número, determine si es par utilizando el operador módulo (Hint: Utilice `%` para determinar el módulo.)
- Utilice una estructura `if-else` para almacenar cada valor en la lista correspondiente mediante `append()`.
- Muestre finalmente el arreglo original, la lista de números pares y la lista de números impares.

Ejercicio 2 – Movimiento parabólico de una pelota

Una pelota es lanzada desde el suelo con una rapidez inicial de

$$v_0 = 20 \text{ m/s}$$

y un ángulo

$$\theta = 45^\circ.$$

Sin considerar la resistencia del aire, la posición de la pelota se puede aproximar mediante

$$x(t) = v_0 \cos(\theta)t,$$

$$y(t) = v_0 \sin(\theta)t - \frac{1}{2}gt^2,$$

donde

$$g = 9.81 \text{ m/s}^2.$$

Escriba un programa que:

- Importe NumPy y Matplotlib.
- Defina las variables `v0`, `angulo` y `g`.
- Convierta el ángulo desde grados a radianes utilizando `np.deg2rad()`.
- Cree un arreglo de tiempos entre 0 y 3 segundos con pasos de 0.1 segundos utilizando `np.arange()`.
- Cree tres listas vacías: `tiempos_validos`, `posicion_x` y `posicion_y`.
- Recorra los tiempos utilizando un ciclo `for`.
- Para cada instante, calcule $x(t)$ e $y(t)$.
- Utilice un `if` para almacenar los resultados solamente cuando la pelota se encuentre sobre el nivel del suelo, es decir, cuando

$$y(t) \geq 0.$$

- Grafique la altura y en función de la posición horizontal x .

Ejercicio 3 – Rectificador ideal de media onda

Un circuito rectificador ideal recibe una señal sinusoidal de entrada

$$v_{\text{in}}(t) = A \sin(2\pi ft),$$

con

$$A = 5 \text{ V}, \quad f = 50 \text{ Hz}.$$

En un rectificador ideal de media onda, la salida se puede representar mediante

$$v_{\text{out}} = \begin{cases} v_{\text{in}}, & v_{\text{in}} \geq 0, \\ 0, & v_{\text{in}} < 0. \end{cases}$$

Escriba un programa que:

- Importe NumPy y Matplotlib.
- Cree un arreglo de tiempo desde 0 hasta 0.04 segundos utilizando `np.arange()`.
- Calcule la señal sinusoidal de entrada.
- Cree una lista vacía llamada `salida`.
- Recorra cada valor de la señal de entrada mediante un ciclo `for`.
- Utilice una estructura `if-else` para almacenar en la lista `salida`:
 - el voltaje de entrada cuando sea positivo;
 - cero cuando sea negativo.
- Grafique en una misma figura la señal de entrada y la señal rectificada.
- Incluya una leyenda que permita identificar ambas señales.

Ejercicio 4 – Modelo sencillo de un LED

Un LED se conecta a una fuente de voltaje mediante una resistencia de

$$R = 220 \, \Omega.$$

Para simplificar el comportamiento del LED, se utilizará un modelo en el que su voltaje de conducción es

$$V_D = 2.0 \, \text{V}.$$

La corriente del circuito se aproximará mediante

$$I = \begin{cases} 0, & V \leq V_D, \\ \frac{V - V_D}{R}, & V > V_D. \end{cases}$$

Se desea estudiar el comportamiento del circuito para diferentes valores del voltaje de alimentación.

- Cree con `np.arange()` un arreglo de voltajes desde 0 hasta 5 V con incrementos de 0.1 V.
- Cree una lista vacía llamada `corrientes`.
- Recorra los voltajes utilizando un ciclo `for`.
- Para cada voltaje utilice una estructura `if-else` para calcular la corriente del circuito.
- Convierta la corriente desde amperes a miliamperes antes de almacenarla en la lista:

$$I_{\text{mA}} = 1000I.$$

- Grafique la corriente del LED en función del voltaje de alimentación.
- Muestre en pantalla la corriente correspondiente a una fuente de 5 V.

Ejercicio 5 – Energía consumida por una carga variable

Un sistema electrónico funciona con una fuente constante de

$$V = 12 \text{ V.}$$

Durante una prueba de 10 segundos, el sistema presenta tres modos de operación:

$$I(t) = \begin{cases} 0.2 \text{ A}, & 0 \leq t < 4 \text{ s}, \\ 0.5 \text{ A}, & 4 \leq t < 7 \text{ s}, \\ 0.1 \text{ A}, & 7 \leq t < 10 \text{ s}. \end{cases}$$

Para intervalos de tiempo de

$$\Delta t = 1 \text{ s,}$$

la energía consumida puede calcularse de forma acumulativa mediante

$$E_{\text{nuevo}} = E_{\text{anterior}} + VI\Delta t.$$

Escriba un programa que:

a) Cree con `np.arange()` los instantes desde 0 hasta 9 segundos.

b) Inicialice la energía acumulada con el valor

$$E = 0.$$

c) Cree tres listas vacías llamadas `instantes`, `corrientes` y `energias`.

d) Recorra los instantes utilizando un ciclo `for`.

e) Determine la corriente correspondiente utilizando una estructura `if-elif-else`.

f) Calcule la potencia mediante

$$P = VI.$$

g) Actualice la energía acumulada mediante

$$E = E + P\Delta t.$$

h) Almacene los resultados utilizando `append()`.

i) Grafique la energía acumulada en función del tiempo.

j) Muestre en pantalla la energía total consumida durante la prueba.

Observación: en este ejercicio el nuevo valor de la energía depende del valor calculado en la iteración anterior. Esta idea será utilizada nuevamente en el ejercicio siguiente.

Ejercicio 6 – Método de Euler: carga y descarga de un capacitor

Considere un circuito RC formado por

$$R = 1000 \, \Omega, \quad C = 1000 \, \mu\text{F}.$$

El voltaje del capacitor cumple la ecuación diferencial

$$\frac{dv_C}{dt} = \frac{V_s(t) - v_C}{RC}.$$

La fuente tiene el siguiente comportamiento:

$$V_s(t) = \begin{cases} 5 \, \text{V}, & t < 3 \, \text{s}, \\ 0 \, \text{V}, & t \geq 3 \, \text{s}. \end{cases}$$

Por lo tanto, durante los primeros 3 segundos el capacitor se carga y, posteriormente, comienza a descargarse.

Para resolver numéricamente la ecuación diferencial se utilizará el **método de Euler**.

Si se conoce el voltaje $v_{C,k}$ en un instante determinado, el siguiente valor se aproxima mediante

$$v_{C,k+1} = v_{C,k} + \Delta t \left(\frac{V_{s,k} - v_{C,k}}{RC} \right).$$

Utilice

$$\Delta t = 0.05 \, \text{s}$$

y considere inicialmente

$$v_C(0) = 0 \, \text{V}.$$

Escriba un programa que:

- Importe NumPy y Matplotlib.
- Convierta la capacitancia desde microfaradios a faradios:

$$C = 1000 \times 10^{-6} \, \text{F}.$$

- Cree un arreglo de tiempo desde 0 hasta 6 segundos.
- Cree una lista llamada `voltaje_capacitor` cuyo primer elemento sea 0.
- Cree una lista vacía llamada `fuentes`.
- Recorra los instantes mediante un ciclo `for`.
- Utilice un `if-else` para determinar el valor de la fuente:
 - 5 V antes de los 3 segundos;
 - 0 V desde los 3 segundos en adelante.

- Calcule la derivada

$$\frac{dv_C}{dt} = \frac{V_s - v_C}{RC}.$$

Ejercicio 6 – continuación

i) Utilice el método de Euler para calcular el nuevo voltaje:

$$v_{C,\text{nuevo}} = v_{C,\text{actual}} + \Delta t \frac{dv_C}{dt}.$$

j) Agregue cada nuevo voltaje a la lista utilizando `append()`.

k) Grafique en una misma figura el voltaje de la fuente y el voltaje calculado para el capacitor.

Consejos

- ejecute cada programa desde el inicio;
- compruebe que no existan errores de ejecución;
- revise que los resultados sean coherentes;
- verifique las unidades utilizadas;
- compruebe que todos los gráficos tengan título, nombres de los ejes y cuadrícula.